Module 11: Defining Workflows to Orchestrate Functions

AWS Academy Cloud Developing

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 1: Introduction

Module 11: Defining Workflows to Orchestrate Functions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module objectives

At the end of this module, you should be able to do the following:

- Recognize the dynamics of workflow coordination in distributed applications
- Describe AWS Step Functions
- Identify state types
- Indicate common use cases for Step Functions
- Recall Step Functions API operations

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module overview

Sections

1. Introduction
2. Coordinating tasks in distributed applications
3. Introducing Step Functions
4. State types
5. Step Functions use cases
6. Step Functions API

Demonstration

- Creating Simple Calculators by Using Step Functions

Lab

- Lab: Orchestrating Serverless Functions with Step Function

aws

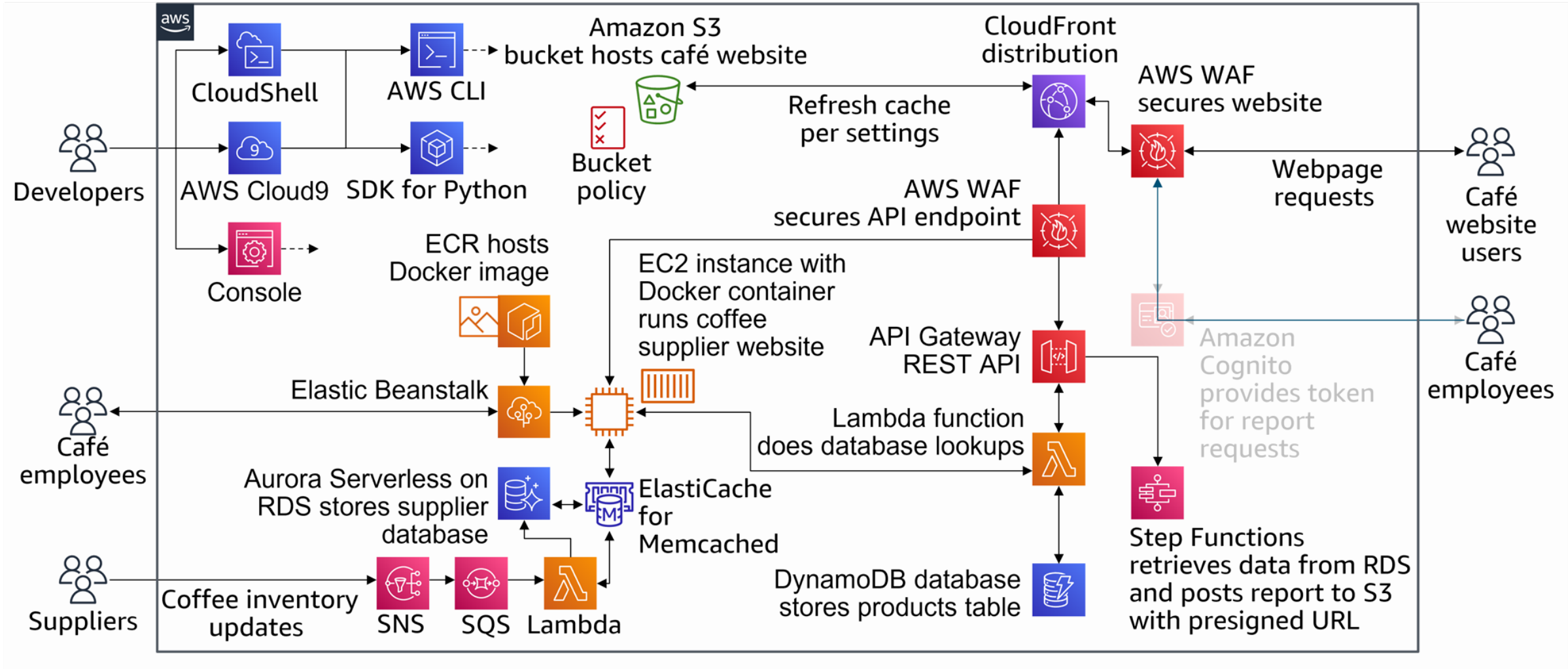
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Café business requirement

Now that the coffee inventory is automatically updated on the café website, Frank would like to be able to request reports with the latest inventory information. Sofía thinks she can use Step Functions to coordinate the steps to generate a report.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Step Functions as part of developing a cloud application



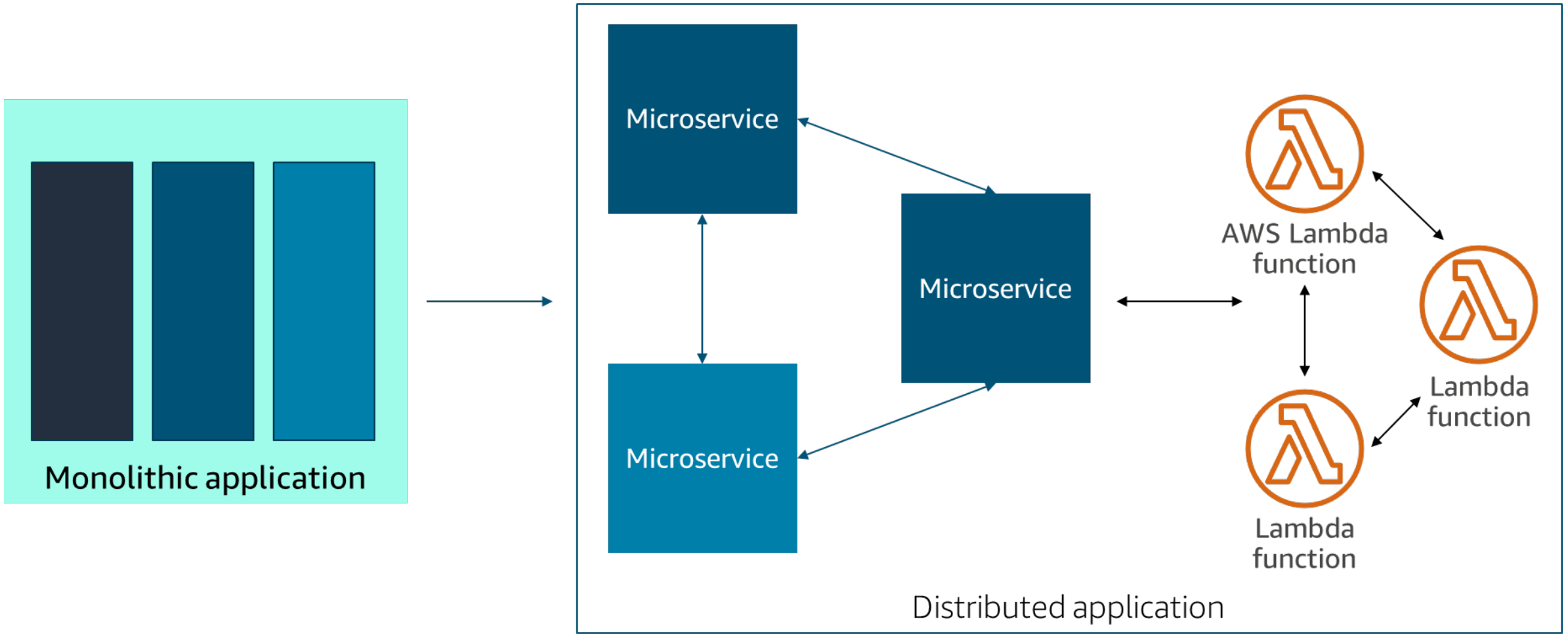
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 2: Coordinating tasks in distributed applications

Module 11: Defining Workflows to Orchestrate Functions

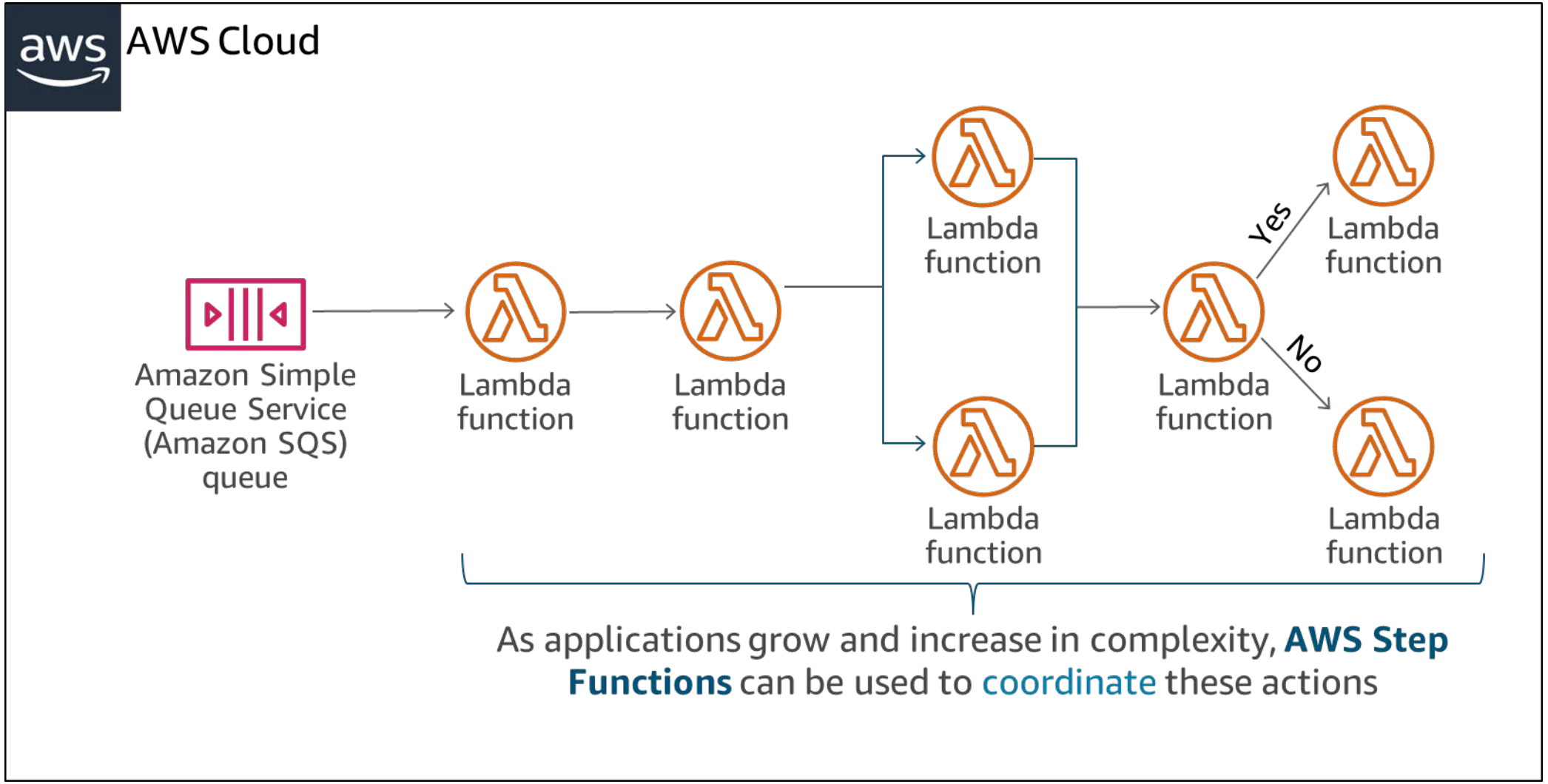
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Microservices architecture for distributed applications



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Challenges with distributed applications



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 2 key takeaways

- **Microservices** are smaller, independent components that make up an application in modern cloud architecture.
- You can use microservices to quickly **deploy, scale,** and **update** application components.
- A **coordination layer** is necessary to handle **automatic scaling, errors,** and **timeouts**.

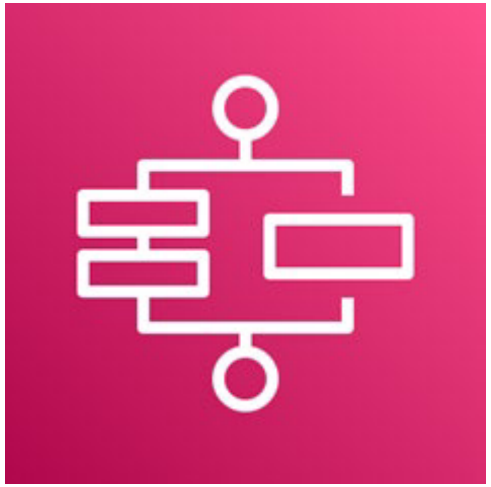
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 3: Introducing Step Functions

Module 11: Defining Workflows to Orchestrate Functions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS Step Functions



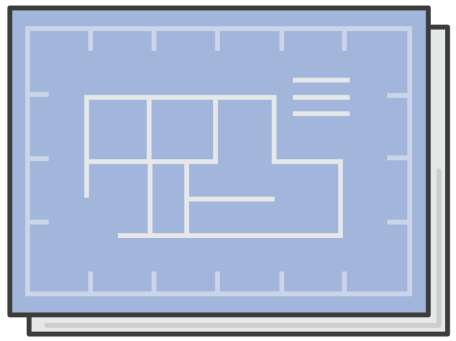
AWS Step Functions

Serverless orchestration service that you can use to combine Lambda functions and other AWS services to build critical applications

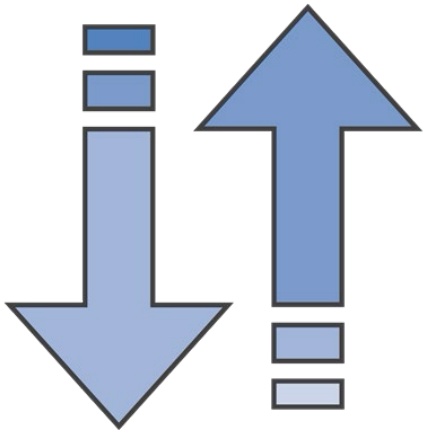
- Coordinates components of distributed applications and microservices by using visual workflows
- Automatically triggers and tracks each step, and retries when errors occur
- Logs the state of each step

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Benefits of Step Functions



Productivity
Build and update applications quickly

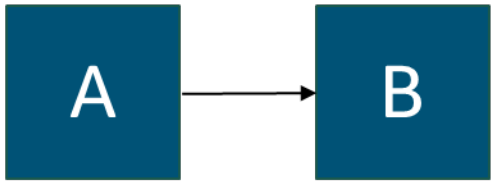


Agility
Scale and recover reliably

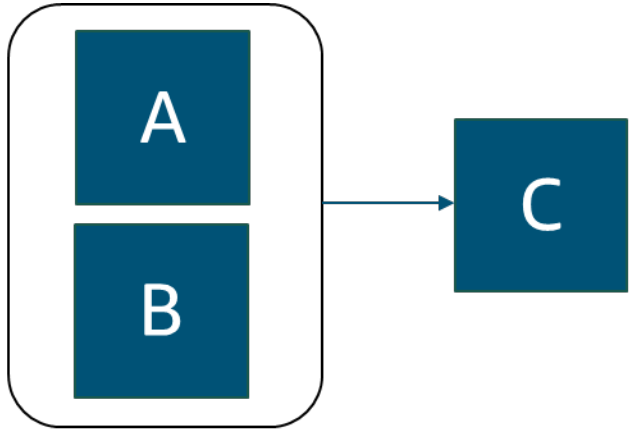
Resiliency
Evolve applications easily

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

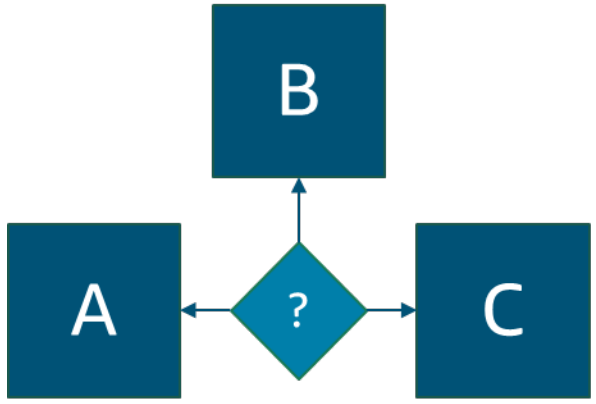
Workflow options



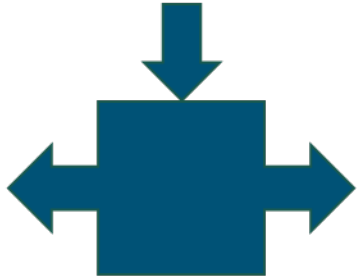
Run tasks in sequence



Run tasks in parallel



Select a task based on data



Manage try-catch-finally behavior

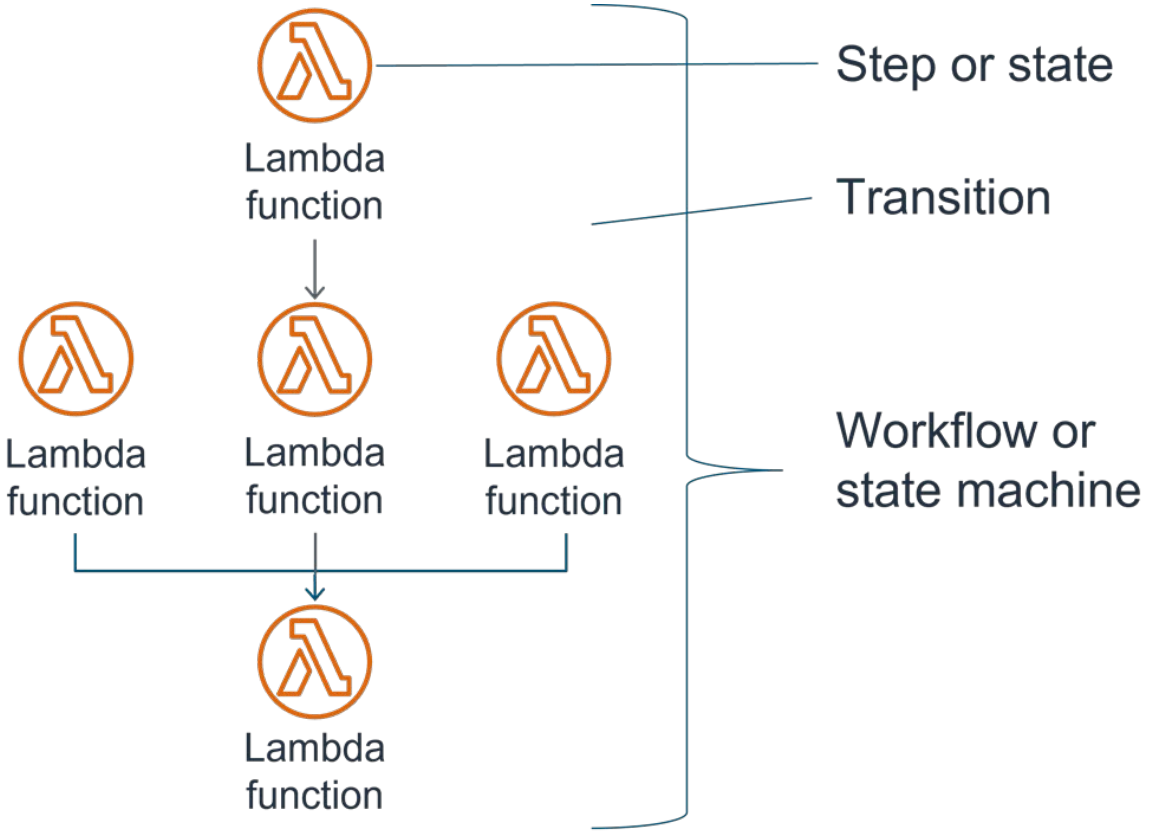
Retry failed tasks

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How Step Functions works

- You define your **workflow** (or state machine) as a series of **steps** (or states) and **transitions**.
- **Tasks** do all of the work in a state machine.
- Each state machine starts with an **input**. The state transforms the input and passes the **output** to the next state.
- You can **visualize** and **track** workflows in the Step Functions console.



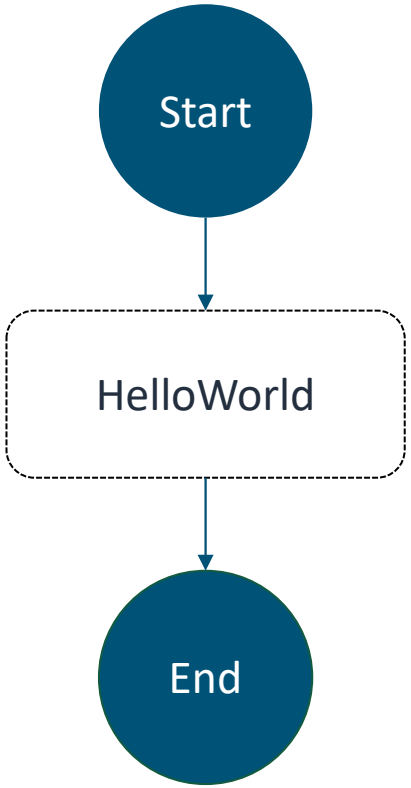
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Defining and visualizing workflows

Define the workflow in JSON by using the Amazon States Language

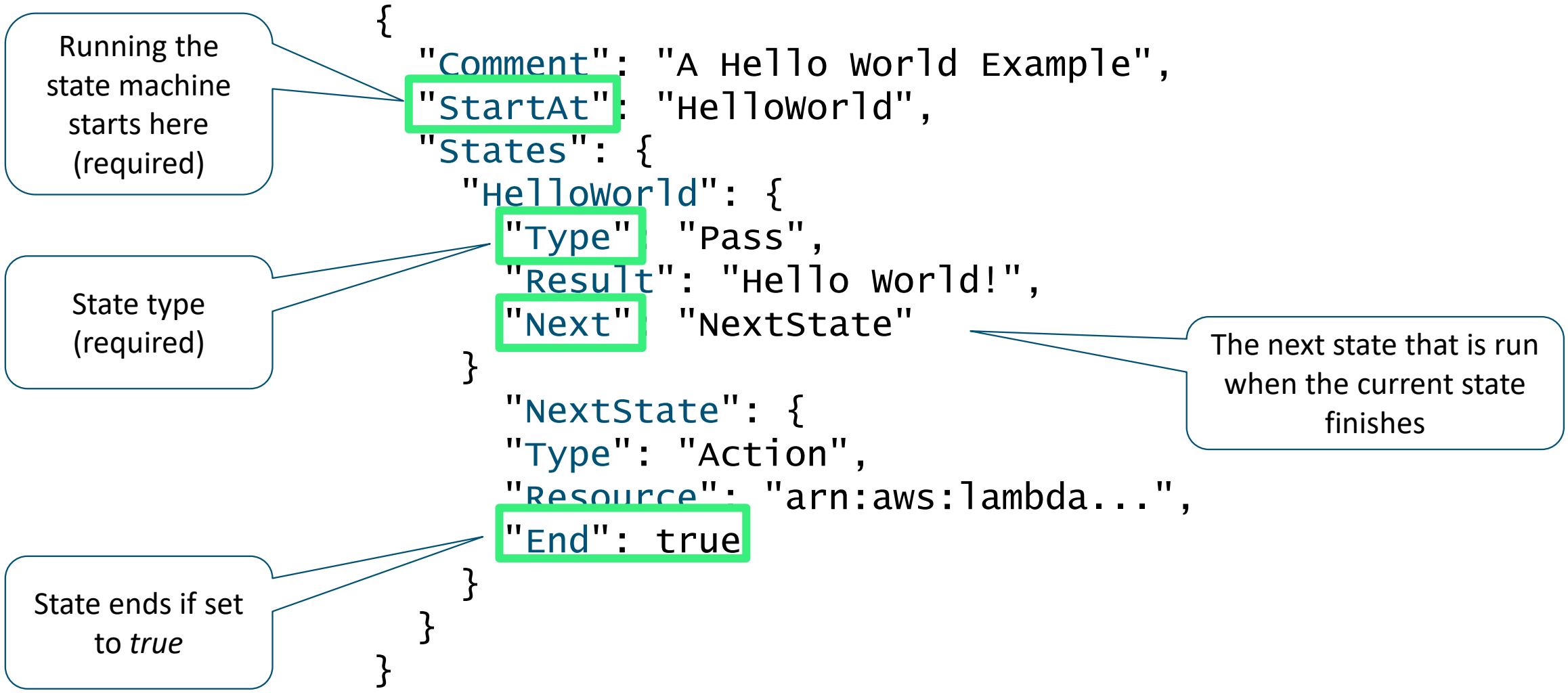
```
{
  "Comment": "A Hello World Example",
  "StartAt": "HelloWorld",
  "States": {
    "HelloWorld": {
      "Type": "Pass",
      "Result": "Hello World!",
      "End": true
    }
  }
}
```

Visualize the workflow in the Step Functions console



aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Running state machines



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 3 key takeaways

- **Step Functions** is a service that you can use to coordinate components of distributed applications and **microservices** by using visual workflows.
- A **state machine** is an object that has a set number of operating conditions that depend on the object's **previous condition** to determine the **output**.
- State machines are **defined** by using the **Amazon States Language**.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 4: State types

Module 11: Defining Workflows to Orchestrate Functions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

State types

Task	A single unit of work
Choice	Adds branching logic to a state machine
Fail	Stops running a state and marks it as a failure
Succeed	Stops running a state successfully
Pass	Passes its input to its output, without performing work
Wait	Delays from continuing for a specified time
Parallel	Creates parallel branches for running states
Map	Dynamically iterates steps

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Task state

Amazon Elastic Compute Cloud (Amazon EC2)

Amazon Elastic Container Service (Amazon ECS)

Lambda function

```
"ActivityState": {
        "Type": "Task",
        "Resource": "arn:aws:states:us-east-1:123456789012:activity:HelloWorld",
        "TimeoutSeconds": 300,
        "HeartbeatSeconds": 60,
        "Next": "NextState"
    }
```

```
"LambdaState": {
        "Type": "Task",
        "Resource": "arn:aws:lambda:us-east-1:123456789012:function:HelloWorld",
        "Next": "NextState"
    }
```

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

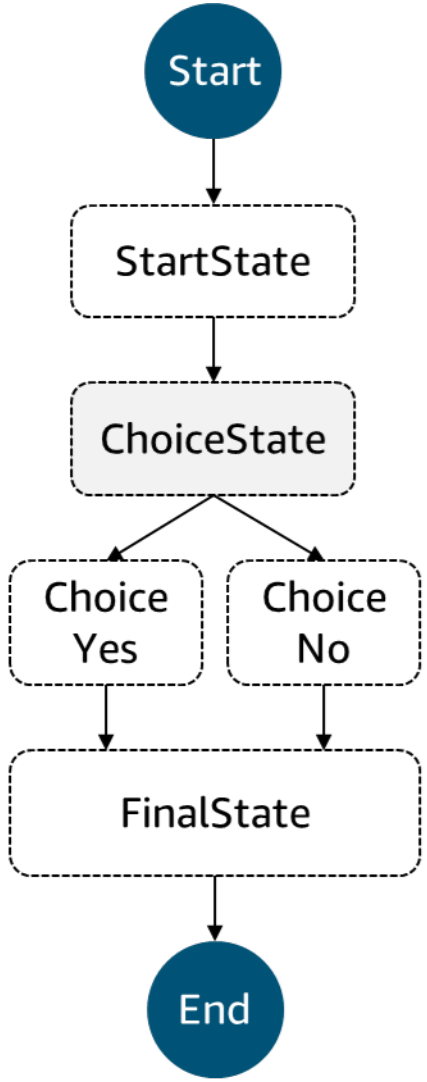
Choice state

```
"ChoiceState": {
      "Type" : "Choice",
      "Choices": [
        {
          "Variable": "$.branch",
          "NumericEquals": 1,
          "Next": “Choice Yes"
        },
        {
          "Variable": "$.branch",
          "NumericEquals": 2,
          "Next": "Choice No"
        }
      ],
      "Default": "DefaultState"
}
```

Choice rule

Choice rule

Required



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Fail and Succeed states

```
"FailState": {
      "Type": "Fail",
      "Cause": "Invalid response.",
      "Error": "ErrorA"
}
```

Fail state

```
"SuccessState": {
      "Type": "Succeed"
}
```

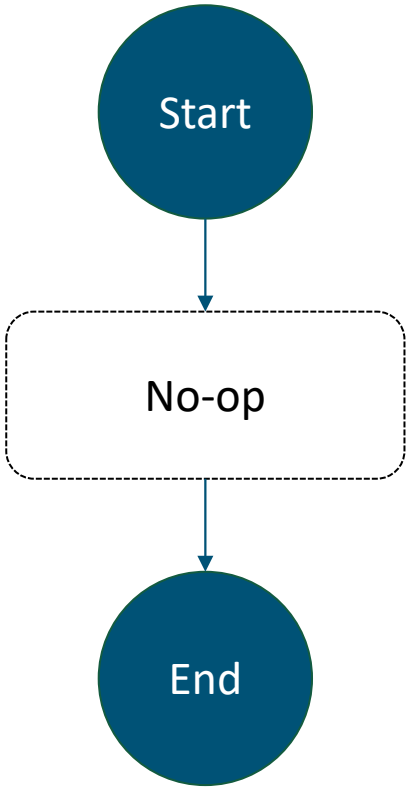
Succeed state

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Pass state

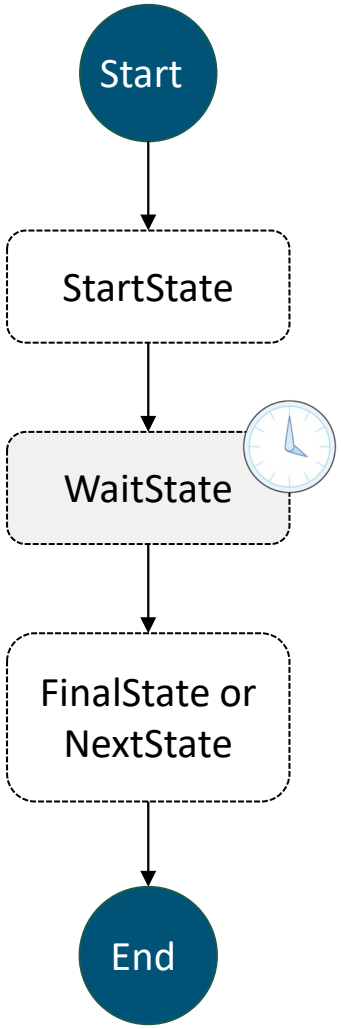
```
"No-op": {
        "Type": "Pass",
        "Result": {
        "x-datum": 0.381018,
        "y-datum": 622.2269926397355
        },
        "ResultPath": "$.coords",
        "Next": "End"
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Wait state

```
"wait_ten_seconds": {
        "Type": "Wait",
        "Seconds": 10,
        "Next": "NextState"
}
```



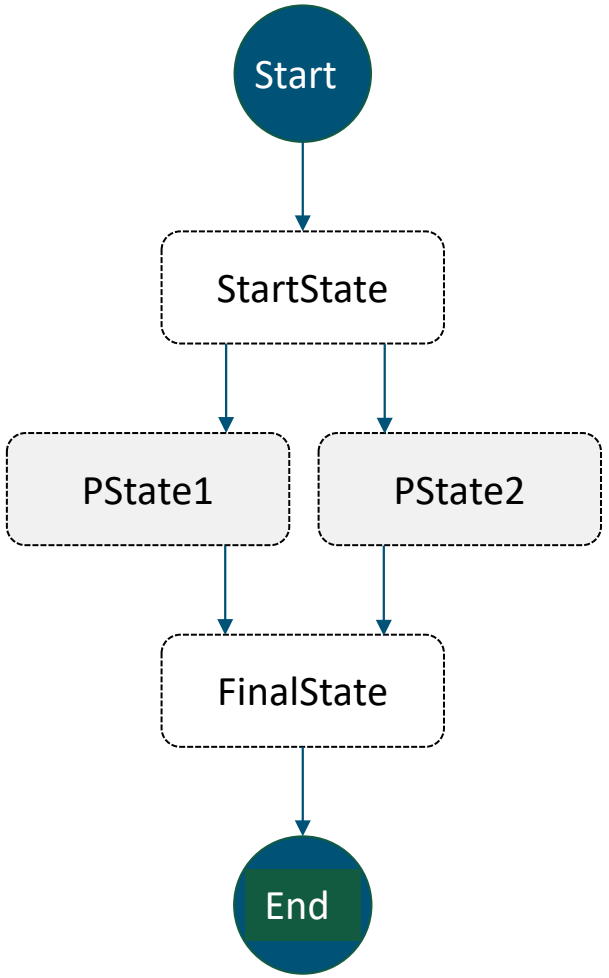
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Parallel state

```
"LookupCustomerInfo": {
        "Type": "Parallel",
        "Branches": [
         {
                "StartAt": "PState1",
                "States": {
                        "PState1": {…}
                }
         },
         {
                "StartAt": "PState2",
                "States": {
                        "PState2": {…}

                }
         }
        ],
        "Next": "NextState"
}
```

Required



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Map state

```
"Validate-All": {
  "Type": "Map",
  "InputPath": "$.detail",
  "ItemsPath": "$.shipped",
  "MaxConcurrency": 0,
  "Iterator": {
    "StartAt": "Validate",
    "States": {
      "Validate": {
        "Type": "Task",
        "Resource": "arn:aws:lambda:us-east-1:123456789012:function:ship-val",
        "End" : true
      }
    }
  },
  "ResultPath": "$.detail.shipped",
  "End": true
}
```

Iterator (required)

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 4 key takeaways

- Individual states can make **decisions** based on their **input**, perform **actions**, and pass and manipulate **output** to other states.
- Add **branching logic** to a state machine by adding a **Choice** state.
- Run multiple functions **concurrently** by using the **Parallel** state.
- **Succeed** and **Fail** states do not require an **End** field.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 5: Step Functions use cases

Module 11: Defining Workflows to Orchestrate Functions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

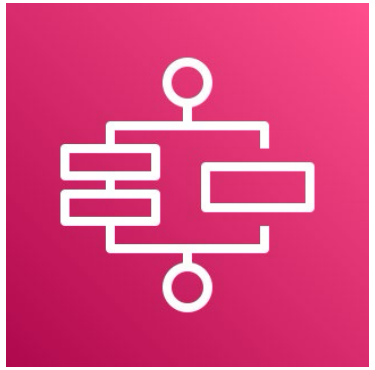
Standard and Express workflows

	Standard	Express
Invocation Rate	2,000 per second	100,000 per second
Transition Rate	4,000 per second	Nearly unlimited
Pricing	Per state transition	Per number and duration of requests for your workflow
Step Functions Activities	Supports Step Functions activities	Does not support Step Functions activities
Integrations and Patterns Support	Supports all service and integration patterns	Supports all service integrations and most patterns

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Standard workflows

- Ideal for long-running, auditable workflows
- Can run for up to **1 year**
- Employ an exactly-once model
- Never run more than **once** unless the Retry behavior is specified in ASL
- Are billed according to the number of processed state transitions



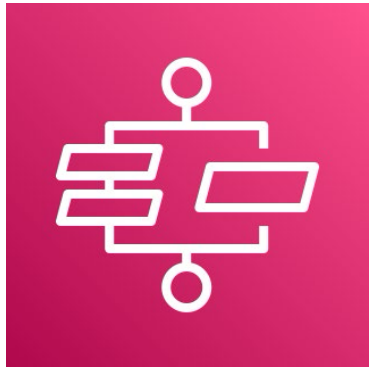
AWS Step Functions

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Express workflows

- Ideal for high-volume, event-processing workloads
- Can run for up to **5 minutes**
- Employ an at-least-once (asynchronous) or at-most-once (synchronous) model
- An invocation can run **more than once**
- Are billed according to number and duration of requests, and memory that is consumed



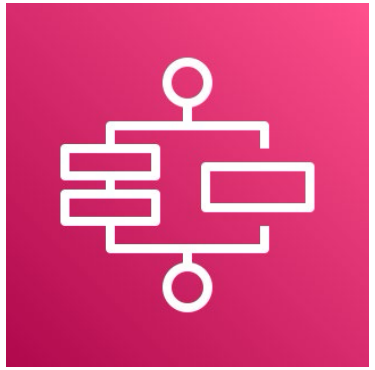
AWS Express Workflows

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Step Functions use cases

- Process data
- Automate tasks
- Modernize a monolithic application
- Orchestrate an application
- Transcode media files
- Sequence batch processing jobs
- Send messages from automated workflows
- Publish events from serverless workflows
- Coordinate container tasks in microservices and serverless applications
- Access databases from serverless workflows
- Sequence steps of machine learning (ML) workflows
- Coordinate extract, transform, and load (ETL) jobs

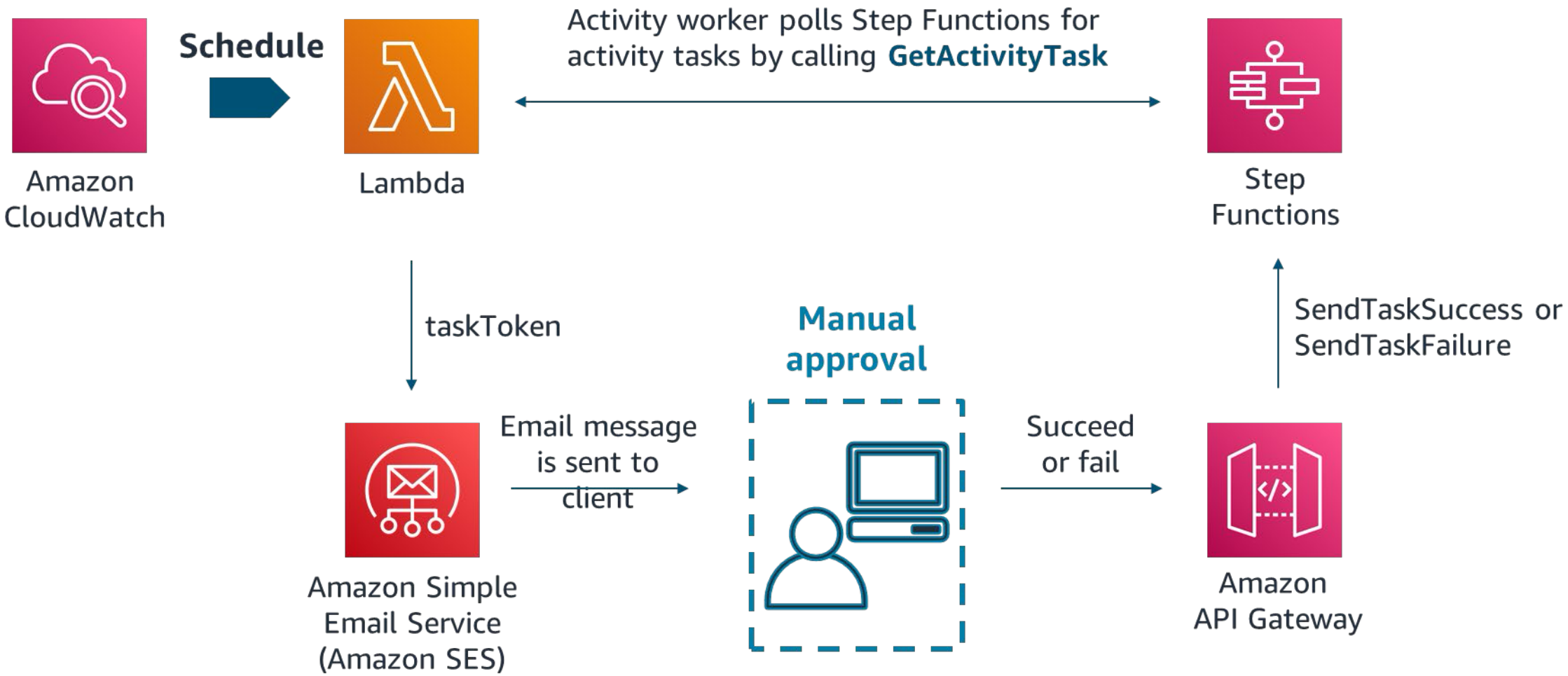


AWS Step Functions

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Step Functions use case example: Manual approval



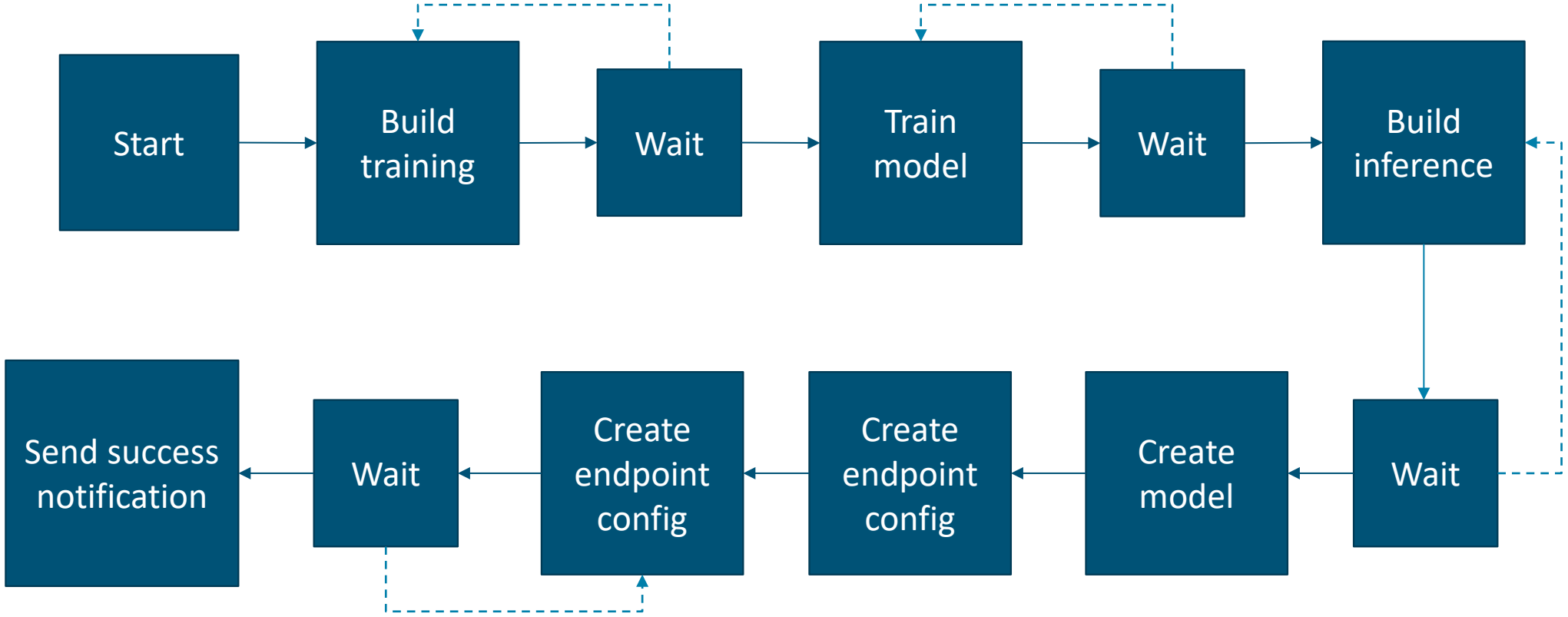
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Use case example (continued)

```
{
    "Comment": "Employee promotion process",
    "StartAt": "ManualApproval",
     "States": {
       "ManualApproval": {
           "Type": "Task",
           "Resource": "arn:aws:states:us-east-1:ACCOUNT_ID:activity:ManualStep",
           "TimeoutSeconds": 3600,
           "End": true
       }
     }
}
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Step Functions use case example: Machine learning workflow



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 5 key takeaways

- **Activities** are a Step Functions feature that **associate code** with a **specific task** in a state machine.
- A **taskToken** is a token that represents a specific task that Step Functions generates when a task is assigned to a worker.
- An activity worker can run on any application that is able to make an **HTTP** connection.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 6: Step Functions API

Module 11: Defining Workflows to Orchestrate Functions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Step Functions API

Create	Upload state machines that are defined in JSON. Register activity workers.
StartExecution	Return an Amazon Resource Name (ARN) that identifies the run.
StopExecution	Return the date that the state machine stopped.
List	List all state machines, runs, and activities.
Describe	Describe individual state machines, runs, and activities.
GetExecutionHistory	Get the history of a state machine run as a list of events.

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Step Functions quotas

- General quotas (for example, names in Step Functions)
- Quotas that are related to:
  - Accounts (for example, maximum number of registered state machines or API actions)
  - State machine runs (for example, maximum runtime)
  - Task runs (for example, maximum task runtime)
  - API action throttling
  - State throttling
- Quotas might differ between Standard and Express workflows

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 6 key takeaways

- You can access Step Functions by using the **AWS Management Console**, **AWS SDKs**, or **API**.
- Step Functions places **quotas** on the size of certain state machine parameters.
- Use **API Gateway** to associate your Step Functions APIs with **methods** in an API Gateway API.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Demonstration: Creating Simple Calculators by Using Step Functions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab 11.1: Orchestrating Serverless Functions with Step Functions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Scenario

In this lab, you will build the functionality to create and deliver the coffee inventory report using Step Functions.

aws

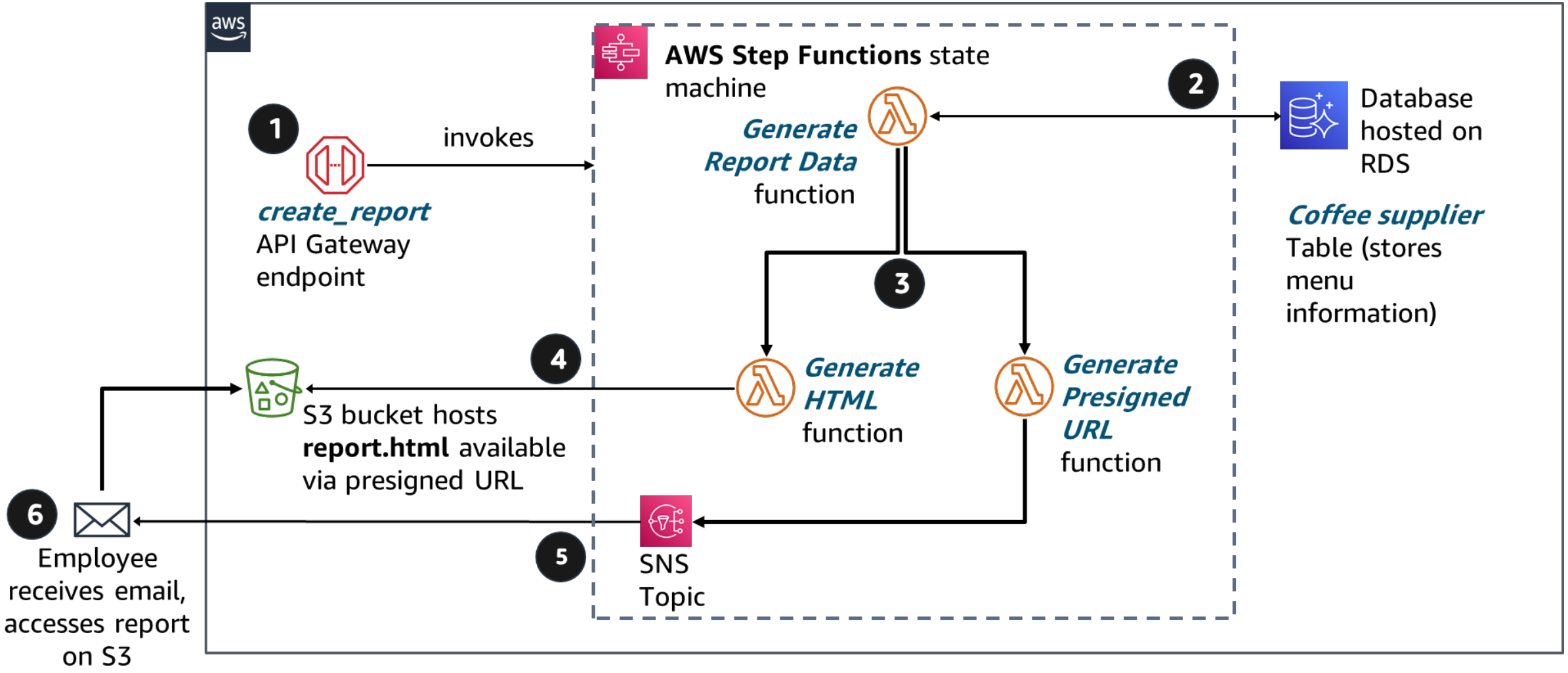
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Tasks

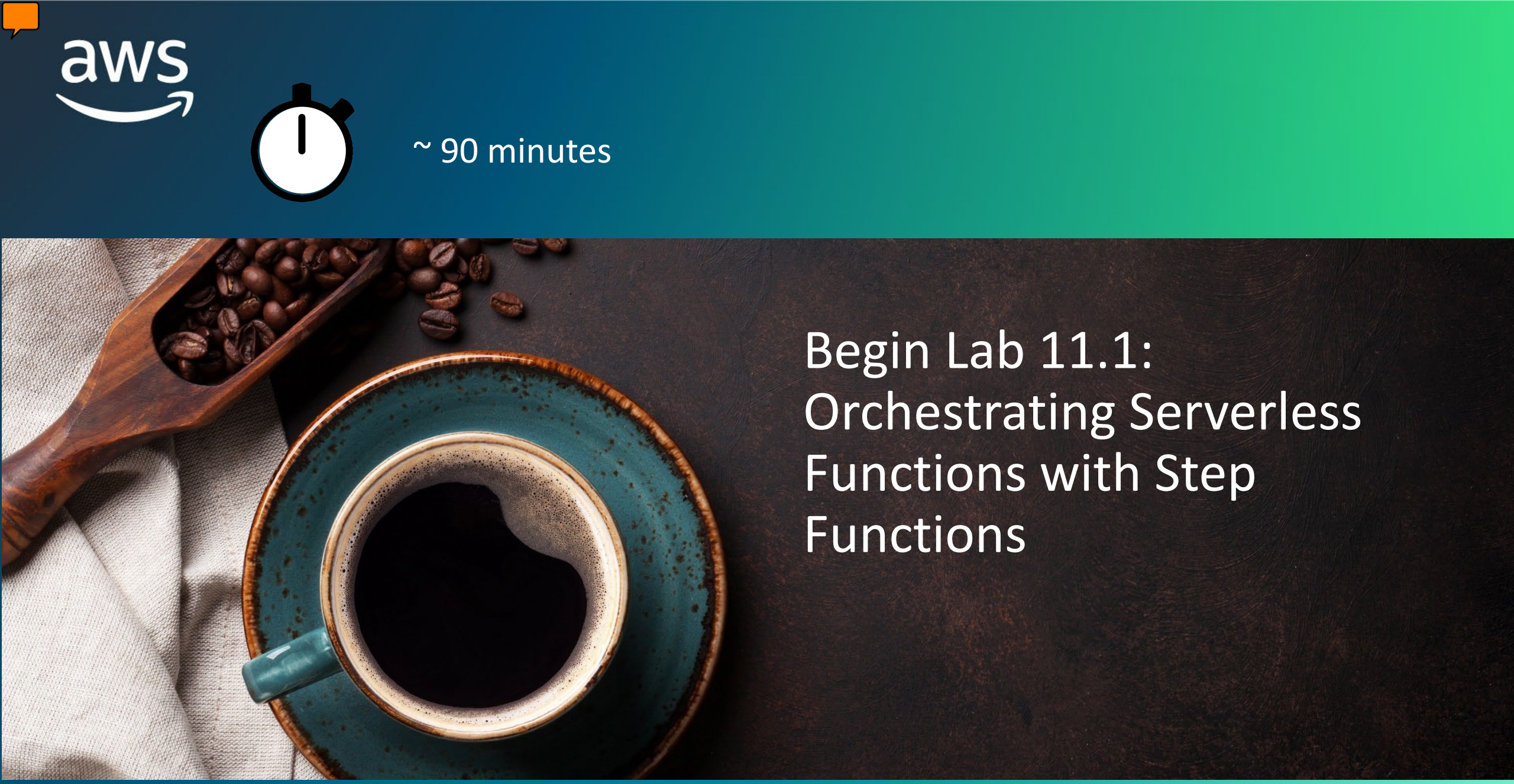
1. Preparing the development environment
2. Configuring an SNS topic and subscribing to it
3. Creating a Step Functions state machine
4. Creating a Lambda function to generate a presigned URL
5. Configuring the REST API to invoke the state machine
6. Configuring a Lambda function to generate an HTML report
7. Adding the GenerateHTML function to the state machine
8. Creating a Lambda function to retrieve supplier records
9. Adding the generateReportData function to the state machine

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Final product



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



~ 90 minutes

Begin Lab 11.1: Orchestrating Serverless Functions with Step Functions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab debrief: Key takeaways

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module wrap-up

Module 11: Defining Workflows to Orchestrate Functions

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module summary

In summary, in this module, you learned how to:

- Recognize the dynamics of workflow coordination in distributed applications
- Describe Step Functions
- Identify state types
- Indicate common use cases for Step Functions
- Recall Step Functions API operations

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Complete the knowledge check

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question

When a customer submits a request for a credit limit increase, the request is to be processed automatically by comparing their request to their credit limit. If the request is more than their preapproved credit limit, Step Functions must send the request to a manager for approval.

Which type of state should be used to accomplish this process?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- **Comparing**
- **Send the request to a manager**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question: Responses

When a customer submits a request for a credit limit increase, the request is to be processed automatically by **comparing** their request to their credit limit. If the request is more than their preapproved credit limit, Step Functions must **send the request to a manager** for approval.

Which type of state should be used to accomplish this process?

Choice	Response
A	Parallel
B	Choice
C	Task
D	Pass

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question: Answer

The correct answer is B.

Choice	Response
A	Parallel
B	**Choice**
C	Task
D	Pass

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Additional resources

- AWS Machine Learning Blog: Automated and continuous deployment of Amazon SageMaker models with AWS Step Functions
- AWS Compute Blog: Implementing Serverless Manual Approval Steps in AWS Step Functions and Amazon API Gateway
- AWS Technical Guide: Create a Serverless Content Syndication Pipeline with AWS Step Functions
- FAQ: AWS Step Functions FAQs

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Thank you

Corrections, feedback, or other questions?
Contact us at https://support.aws.amazon.com/#/contacts/aws-academy.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.